

Efficient Optimization of Fixed-Length Paths

Martino Ciaperoni¹, Nikolaos Tziavelis² and Panagiotis Karras³

¹Scuola Normale Superiore, Pisa, Italy

²UC Santa Cruz, USA

³University of Copenhagen, Denmark

martino.ciaperoni@sns.it, ntziavel@ucsc.edu, piekarras@gmail.com

Abstract

Optimization problems such as *Viterbi decoding* and *V-optimal histogram* construction seek a path of *exact* length L through a state space that minimizes a cost function. These problems are traditionally solved using dynamic programming (DP). A best-first-search (BFS) solution is also applicable, yet requires maintaining a priority queue. In all cases, memory usage grows linearly with both state space size and path length. In this paper, we propose COMPACTBFS, a framework that limits the growth of the BFS priority queue to space-efficiently determine the exact optimal cost for a fixed-length path and then constructs such a path by a divide-and-conquer strategy that eliminates the memory overhead. We apply COMPACTBFS to Viterbi decoding, which remains relevant to speech recognition, and V-optimal histogram construction. Our experimental results demonstrate significant gains over state-of-the-art solutions in runtime and memory consumption.

1 Introduction

Several problems call for finding a sequence of given length L over a space of n states and m connections that optimizes a cost function; they are conventionally solved by A* search or dynamic programming (DP). Examples include *Viterbi decoding*, i.e., finding a most likely sequence of hidden states—a *Viterbi path*—in a Hidden Markov Model (HMM) to explain an observed event sequence of length L [Viterbi, 1967, 2006], prominently used in speech recognition [Pratap *et al.*, 2024]; finding a sequence of $L-1$ boundaries—a *V-optimal histogram* (‘V’ for variance)—that partitions a numeric sequence minimizing the total squared error of representing each partition by a constant value [Jagadish *et al.*, 1998]; and identifying the most probable syntactic structure—a Viterbi parse tree—that generates a length- L input sequence of tokens or words under a probabilistic context-free grammar (PCFG) [Klein and Manning, 2003; Huang *et al.*, 2012].

Such algorithms iteratively derive solutions to a subproblem of length $k+1$ from solutions to subproblems of length k in $\mathcal{O}(mL)$ time. A*-based algorithms maintain a priority queue to prioritize subproblems that aid the solution in a *best-first* manner with an $\mathcal{O}(nL \log nL)$ complexity overhead,

while DP prioritizes them in a *breadth-first* manner by length. Both *memoize* partial solutions; after determining the optimal cost, they backtrack over those memoized solutions to retrieve the path having that cost. In the *shortest-path* problem, path length is *arbitrary* [Russell and Norvig, 2010]; therefore, a lower-cost arrival at node i in step ℓ dominates a higher-cost arrival in a different step ℓ' . Consequently, it suffices to track, for each node, only the minimum arrival cost and the incoming neighbor from which it is reached, using $\mathcal{O}(n)$ space. However, when path length is *predetermined* [Klein and Manning, 2003; Huang *et al.*, 2012] as L , arrivals at the same node i in different steps face continuations of different costs. We should therefore memoize the lowest cost $\text{Opt}(i, \ell)$ of arriving at node i in each step ℓ . Thus memoization requires $\mathcal{O}(nL + m)$ space, leading to memory constraints in modern GPU-based speech recognition [Pratap *et al.*, 2024]. We might instead discard partial solutions to work *in-place* in $\mathcal{O}(n + m)$ space. Still, in that case we need to recursively rerun the algorithm from scratch $L - 1$ times to build the optimal path after finding its cost, incurring $\mathcal{O}(n^2 L^2)$ *time complexity*. A reformulation of the DP solution [Ciaperoni *et al.*, 2022, 2024] constrains space to $\mathcal{O}(n + m)$ and runs in $\mathcal{O}(n^2 L \log L)$ time, yet does not straightforwardly apply when prioritizing subproblems by *best-first* search using a priority queue.

In this paper, we introduce COMPACTBFS, a framework for space- and time-efficient optimization of *fixed-length* sequences. COMPACTBFS controls the priority queue size by novel strategies we propose, and avoids memoization through divide-and-conquer, yielding $\mathcal{O}(n + m)$ space use (where m stands for the in-memory model). We apply COMPACTBFS to Viterbi decoding and V-optimal segmentation and test it on real and synthetic data, showing gains in memory use and runtime vs. prior work, especially under skewed cost distributions.

2 Background and Related Work

Dynamic programming (DP) [Bellman, 1966] explores the problem space exploiting an *optimal substructure* property, by which a globally optimal solution can be assembled from locally optimal solutions, to solve a problem by recursively *expanding* partial solutions in a breadth-first manner. **Best-first-search** algorithms [Pearl, 1984] may also exploit optimal substructures, as dynamic programming does [Sniedovich, 2006], to find a minimum-cost path from a start to an end node by repetitively *expanding*, i.e., visiting the neighbors

of, the *most promising* unexpanded visited node. The A* algorithm [Russell and Norvig, 2010; Foad *et al.*, 2021], an instance of best-first-search, finds a path of *arbitrary* length (i.e., number of steps) that minimizes a cost function, prioritizing paths by a cost estimation heuristic that is *admissible* (i.e., never overestimates a path’s cost) and *consistent* (i.e., never estimates the cost via an intermediary node as lower) [Russell and Norvig, 2010]; the search may proceed from both start and end by bidirectional search [Pohl, 1969]. A* generalizes Dijkstra’s algorithm [1959], by which the heuristic cost estimate of any unexplored path is 0. Dijkstra retains only the cheapest arrival at each node i and returns the cheapest path upon reaching a certain goal state, length L , or cost [Bemten *et al.*, 2019]; it is based on the assumption that any continuation from node i adds the same cost to all paths. However, when seeking a minimum-cost path of exactly L steps, arrivals at the same node i at different steps ℓ and ℓ' face different feasible continuations, whose cost depends on the exact steps remaining. In effect, a lower-cost arrival at i does not dominate a higher-cost arrival at a different step. An algorithm retaining only the cheapest arrival at each node i may thus miss the globally optimal length- L path; instead, we must track the lowest cost $\text{Opt}(i, \ell)$ of arriving at node i at each step ℓ .

Hidden Markov Models (HMMs) explain observation sequences. An HMM comprises a set of K hidden states, each with probabilities to be an initial state, transition to other states, and emit an observation. *Decoding* seeks a sequence of states most likely to generate a sequence of observations:

Problem 1 (Decoding). *Given an HMM and a sequence of T observations $Y = \{y_1, y_2, \dots, y_T\}$, find the sequence of hidden states $Q = \{s_1^*, s_2^*, \dots, s_T^*\}$ that maximizes the likelihood $P(Q, Y)$.*

The Viterbi algorithm [1967; 2006] solves Problem 1 by DP; it finds application from telecommunications [Viterbi, 2006] to speech recognition [Gales and Young, 2007; Braun *et al.*, 2020; Pratap *et al.*, 2024], where it finds the most probable transcription for an acoustic signal, and, in forced alignment, aligning transcriptions to audio recordings, running on a composition of HMMs in which states represent words and their phonemes. However, it raises high memory and runtime requirements. Recently, Jo *et al.* [2019] proposed a heuristic for word recognition without proving its correctness. Ciaperoni *et al.* [2022, 2024] enhanced decoding space efficiency at the cost of a runtime overhead via *divide-and-conquer*. A similar strategy has been applied to *arbitrary-length* sequence alignment [Hirschberg, 1975, 1977; Korf, 1999; Korf and Zhang, 2000; Spouge, 1989, 1991], but not to fixed-length path optimization, as in selecting and aligning L characters from each of two input strings. Other works reduce the state space representation for HMM classes [Siddiqi and Moore, 2005; Felzenszwalb *et al.*, 2003]. Improving on time complexity is a tough undertaking, due to lower bounds [Backurs and Tzamos, 2017]. Klein and Manning [2003] and Huang *et al.* [2012] apply A*-like policies to enhance Viterbi’s time efficiency for PCFG parsing, yet neglect space efficiency. In contrast, we offer a time-efficient and parsimonious formulation.

Histogram construction splits a data series into a number of representative buckets that minimize an error function:

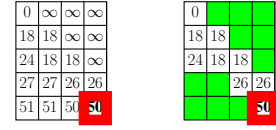


Figure 1: Cells stand for subproblems; DP solves all subproblems; COMPACTBFS does not need to solve subproblems in green.

Problem 2 (Histogram Construction). *Given $I = \{x_1, \dots, x_n\}$, $x_i \in \mathbb{R}$, and $B \in \mathbb{Z}^+$, find a segmentation (or histogram) H_B of I into B non-overlapping subsequences (or buckets) I_b with associated bucket representatives $\hat{x}_b$, $b \in \{1, \dots, B\}$, that minimizes error function $E_I(H_B)$.*

We focus on *V-optimal* histograms [Jagadish *et al.*, 1998], i.e., $E_I(H_B) = \sum_{b=1}^B E_b$, where $E_b = \sum_{x_i \in I_b} (x_i - \hat{x}_b)^2$, and $\hat{x}_b$ is the mean of values in bucket I_b . This problem is solved by a DP algorithm [Jagadish *et al.*, 1998; Guha *et al.*, 2006; Halim *et al.*, 2009] quadratic in n . COMPACTBFS offers gains extensible to any monotonic and distributive error measure [Karras and Mamoulis, 2008].

Semirings and Dioids [Gondran and Minoux, 2008] A *semiring* is a 5-tuple $(D, \oplus, \otimes, \bar{0}, \bar{1})$, where D is a non-empty set, $\oplus$ is a binary, associative, and commutative operator, $\otimes$ is a binary and associative operator, $\bar{0}$ is a neutral element for $\oplus$ (i.e., $x \oplus \bar{0} = x$, for all $x \in D$), $\bar{1}$ is a neutral element for $\otimes$ (i.e., $x \otimes \bar{1} = \bar{1} \otimes x = x$, for all $x \in D$), the operator $\otimes$ distributes over $\oplus$ and $\bar{0}$ is absorbing for $\otimes$ (i.e., $x \otimes \bar{0} = \bar{0} \otimes x = \bar{0}$, for all $x \in D$). A *selective dioid* is a semiring in which $\oplus$ is also selective (i.e., $(x \oplus y = x) \vee (x \oplus y = y)$, for all $x, y \in D$). Selective dioids provide an abstract expressive framework for shortest-path and DP problems [Mohri, 2002; Huang, 2008; Tziavelis *et al.*, 2025].

3 The COMPACTBFS Framework

We define COMPACTBFS and apply it to Viterbi decoding (§ 3.1) and histogram construction (§ 3.2). COMPACTBFS finds a fixed-length sequence that optimizes a cost measure. Unlike dynamic programming, which visits subproblems in a fixed order even if several of them do not contribute to the final solution, COMPACTBFS solves subproblems in a best-first fashion. Figure 1 shows an example.

COMPACTBFS starts out with the following components:

1. A *data space* $\mathcal{X}$ of n elements;
2. A *neighbor* function $\mathcal{N}: \mathcal{X} \rightarrow 2^{\mathcal{X}}$ that generates a *set* $\mathbb{X}$ of length- L *eligible* sequences (x_{i_ℓ}) , $x_{i_\ell} \in \mathcal{X}$, $\ell \in \{1, \dots, L\}$, with $x_{i_{\ell-1}} \in \mathcal{N}(x_{i_\ell})$ for $\ell \in \{2, \dots, L\}$.
3. A *gap function* $G(j, i, \ell)$ associating a value with the transition from item $x_j \in \mathcal{X}$ to item $x_i \in \mathcal{X}$ at step ℓ .
4. A selective dioid $\mathcal{D} = (D, \oplus, \otimes, \bar{0}, \bar{1})$ expressing the *value* of a sequence: $f(\mathbf{x}) = \bigotimes_{\ell=2}^L G(x_{i_{\ell-1}}, x_{i_\ell}, \ell)$, whereby extending a sequence never improves its value.
5. A *problem* that seeks an eligible length- L sequence $\mathbf{x}^* \in \mathbb{X}$ of *optimal* $f(\mathbf{x}^*)$, compared via the $\oplus$ operator.
6. A *recursive function* $\text{Opt}(i, \ell)$ that stores the *optimal* value for an eligible length- ℓ sequence ending at $x_i \in \mathcal{X}$.
7. A *solution* to the problem in Item 5 by DP over sequences of increasing length from $\mathcal{X}$.

The solution in Item 7 finds an eligible sequence of L data items $\mathbf{x}^* = \{x_1^*, x_2^*, \dots, x_L^*\}$ that optimizes $\text{Opt}(\cdot, L)$; the selective dioid properties, in particular distributivity, guarantee correctness. The DP computation takes the form:

$$\text{Opt}(i, \ell) = \bigoplus_{j \in \mathcal{N}(i)} \{\text{Opt}(j, \ell - 1) \otimes G(j, i, \ell)\} \quad (1)$$

The recursion of Equation (1) requires $\Theta(n^2 L)$ time and $\Theta(nL)$ space, iterating over items i and lengths ℓ and storing, for each (i, ℓ) pair, a predecessor needed to backtrack the optimal sequence. The entailed solution may be implemented by either DP or *token passing* [Young *et al.*, 1989]; both calculate *all* solutions of length ℓ , $\text{Opt}(\cdot, \ell)$, before those of length $\ell + 1$, $\text{Opt}(\cdot, \ell + 1)$, by *breadth-first search*. While DP draws from solutions of length ℓ to build each solution of length $\ell + 1$, token passing broadcasts a *token* for each solution of length ℓ to its continuations of length $\ell + 1$. In both cases, solutions solidify at length ℓ before moving to length $\ell + 1$.

COMPACTBFS abolishes this *breadth-first* orientation in favor of a *best-first* one; it organizes sub-problem solutions (represented by tokens) in a priority queue $\mathbf{Q}$ in which it initially inserts all tokens $(\cdot, 1)$. Thereafter, in each step, it selects the most promising token (h, ℓ) from $\mathbf{Q}$ and updates the priority in $\mathbf{Q}$ of each eligible successor $(i, \ell + 1)$ not already extracted from $\mathbf{Q}$ to $\text{Opt}(h, \ell) + G(h, i, \ell)$, when beneficial relative to the existing value. COMPACTBFS resembles Dijkstra’s shortest-path algorithm [1959]; however, whereas Dijkstra minimizes a cost objective regardless of length (i.e., number of steps), COMPACTBFS optimizes the objective under a *fixed-length* constraint. In the worst case, it examines all sequence continuations for each length. However, in practice, it gains performance as it quickly derives tokens corresponding to DP table cells without considering all possible paths and does not produce some tokens at all. As we will see in Section 4, this pruning capacity results in significant savings, particularly in real problem instances. In problem-tailored COMPACTBFS variants, we anticipate the cost a sequence may obtain as it expands and prioritize tokens accordingly. We also derive bidirectional-search variants that produce both prefixes and suffixes of sequences until they reach the target length.

In the best case, COMPACTBFS may produce only L tokens. Nevertheless, the priority queue $\mathbf{Q}$ may reach size $\Theta(nL)$, holding one token for each state-length pair. Additionally, we need to memoize the subproblem solution each token represents even after we pop it from $\mathbf{Q}$, to enable *backtracking* the solution sequence. To ensure $\mathcal{O}(n)$ space complexity, which is highly desirable [Pratap *et al.*, 2024], we employ the following measures, which still calculate the same exact objective.

Controlling the priority queue size. First, to keep the size of $\mathbf{Q}$ in $\mathcal{O}(n)$, when the best-first-search queue $\mathbf{Q}$ exceeds a predefined size threshold θ (or memory budget) upon inserting a token (s_j, t) , we invoke a *containment mechanism*, which is either a tailored *depth-first-search* (DFS) mechanism, or a mechanism that we propose, *earliest-first search* (EFS).

DFS. By the DFS mechanism, we pick the lowest-cost token (s_j, t^*) from the set of tokens for s_j , $\mathcal{S}^j = \{(s_j, \cdot)\}$, and generate all its derivative tokens via a DFS traversal of the HMM graph G starting at s_j . Each DFS branch terminates upon reaching the last frame (i.e., step) or upon injecting into

the token set of a state $s_{j'}$ a token that either displaces or yields to a pre-existing one, without increasing memory usage. This DFS mechanism ensures queue size $\mathcal{O}(n)$ by constraining the size of $\mathbf{Q}$ within θ ; it incurs $\mathcal{O}(n^2 L^2)$ time in the worst case, as it may perform a DFS over all $\mathcal{O}(nL)$ tokens for each token; in practice we do not encounter a large overhead.

EFS. Small θ values in the DFS mechanism may cause excessive DFS calls, reprocessing the same tokens multiple times and slowing runtime, even compared to that of regular dynamic programming which processes all nL tokens. To address this predicament, we propose an alternative search mechanism that processes each token at most once, *Earliest-First Search* (EFS). EFS processes tokens in increasing time frame order, starting from the earliest frame represented in the priority queue, until the gap between the earliest and latest time frames in the queue falls below a user-specified constant $\Delta_{\mathbf{Q}}$. In effect, the queue size is bounded by $\Delta_{\mathbf{Q}} n$ and best-first search resumes. While the EFS mechanism generates and must store additional tokens, at most $2n - 1$ additional tokens will co-exist in the queue, occupying the earliest time frame and its successor. In effect, EFS ensures queue size $\mathcal{O}(n)$ and retains the compact $\mathcal{O}(nL(n + \log L))$ time including the queue overhead.

Reconstructing the optimal path. Second, instead of memoizing subproblem solutions for backtracking the final solution path, we construct that path by a *divide-and-conquer* strategy as in [Binder *et al.*, 1997; Ciaperoni *et al.*, 2022, 2024]. Upon reaching the *middle* frame of a solution path, we record, with each subsequent token, the edge at that middle frame (or *middle pair*). After establishing the last-frame optimal cost, we recursively rerun the algorithm on the $L/2$ -hop predecessors and successors of that token’s middle pair to construct the entire path, retrieving middle pairs as in an in-order tree traversal [Ciaperoni *et al.*, 2022]. The main DP operation takes $\mathcal{O}(n^2 L)$ time; maintaining the priority queue incurs an $\mathcal{O}(nL \log nL)$ overhead and the divide-and-conquer iteration brings an $\log L$ factor, hence the dominant terms are $\mathcal{O}(n^2 L \log L + nL \log^2 L)$, with $\mathcal{O}(n)$ space.

3.1 The MINT algorithm

The Viterbi algorithm selects a sequence of T states $Q = \{s_1^*, s_2^*, \dots, s_T^*\}$ from a universe of K HMM states $S = \{s_1, s_2, \dots, s_K\}$ connected via a set of edges E , *most likely* to have generated a sequence of T observations $Y = \{y_1, y_2, \dots, y_T\}$. Q is called *Viterbi path*. By the Markov property, the likelihood to be in a state depends only on the previous state. The algorithm thus uses the DP recursion:

$$\begin{aligned} \mathbf{T}[s_i, 1] &= \pi_{s_i} \cdot B_{s_i, y_1}, \\ \mathbf{T}[s_i, t] &= \max_{s_h \in \mathcal{N}_{in}(s_i)} \{\mathbf{T}[s_h, t - 1] \cdot A_{s_h, s_i}\} \cdot B_{s_i, y_t} \end{aligned} \quad (2)$$

where $\mathbf{T}[s_i, t]$ stores the probability of the most likely path ending at state s_i in t steps, or *time frames*, $\mathcal{N}_{in}(s_i)$ is the set of in-neighbors of s_i , π_i is the initial probability of s_i , A_{s_h, s_i} is the probability of transiting from state s_h to state s_i on a directed graph G capturing eligible transitions in the HMM, and B_{s_i, y_t} is the probability of observing y_t at state s_i . This setting suits COMPACTBFS as follows:

1. The *data space* $\mathcal{X}$ is the HMM of K hidden states $S = \{s_1, s_2, \dots, s_K\}$.

2. The *neighbor* function $\mathcal{N}_{in} : \mathcal{X} \rightarrow 2^{\mathcal{X}}$ generates a *set* $\mathbb{X}$ of *eligible* sequences (x_{it}) , $x_{it} \in \mathcal{X}$, $t \in \{1, \dots, T\}$ as length- T paths of consecutive states in the HMM.
3. The *gap function* $G(j, i, t)$ is $A_{s_j, s_i} B_{s_i, y_t}$, or, in log-probabilities, as $\log A_{s_j, s_i} + \log B_{s_i, y_t}$.
4. The selective dioid is $([0, 1], \max, \cdot, 0, 1)$, or, in the domain of log-probabilities, $([-\infty, 0], \max, +, -\infty, 0)$. Thus, the *value function* f assigns probabilities to paths, given the sequence of observations $Y = \{y_1, y_2, \dots, y_T\}$; the probability that Y is generated by a sequence of hidden states $Q = \{s_1, s_2, \dots, s_T\}$ is $P(Q, Y) = \pi_{s_1} \cdot B_{s_1 y_1} \prod_{t=2}^T A_{s_{t-1} s_t} \cdot B_{s_t y_t}$, where $\pi(s_1)$, $A_{s_{t-1} s_t}$, and $B_{s_t y_t}$ are defined as above.
5. The *problem* seeks an eligible sequence of states Q of length T that best explains the given sequence of observations Y , i.e., maximizes probability ($\oplus := \max$).
6. The recursive *function* $\text{Opt}(i, \ell)$ that stores the *optimal* value for an eligible sequence of length ℓ that ends at data item $x_i \in \mathcal{X}$ is the function $\mathbf{T}[s_i, t = \ell]$.
7. The *solution* by DP over sequences of increasing length from $\mathcal{X}$ is given by Equation (2).

The recursion of Equation (2) requires $\mathcal{O}(K^2 T)$ time and $\mathcal{O}(KT)$ space, as it iterates over states s_i and time frames t . For the sake of efficiency and accuracy, we replace products of likelihoods by sums of log-likelihoods. In case the structure of G is known, we iterate only over states s_h that link to state s_i , hence visit each HMM graph edge only once; then time complexity becomes $\mathcal{O}((K + |E|)T)$.

The Viterbi algorithm and its *token passing* variant [Young *et al.*, 1989] operate by *breadth-first search*. MINT (Time Efficient Viterbi) replaces this strategy with *best-first search*. It organizes partial solutions in a priority queue $\mathbf{Q}$ and expands the most promising one in each step, as in [Klein and Manning, 2003]. Like Viterbi, it derives $\mathbf{T}[s_h, t + 1]$ entries from $\mathbf{T}[s_h, t]$ entries. Still, while Viterbi calculates *all* $\mathbf{T}[\cdot, t]$ entries before $\mathbf{T}[\cdot, t + 1]$ entries, MINT first inserts all tokens $(s_h, 1)$ in $\mathbf{Q}$ and then iteratively pops the most promising token (s_h, t) from $\mathbf{Q}$ and, for each *outgoing* neighbor $s_i \in \mathcal{N}_{out}(s_h)$ such that $(s_i, t + 1)$ has not been extracted from $\mathbf{Q}$, it computes $\mathbf{T}[s_h, t] \cdot A_{s_h, s_i} \cdot B_{s_i, y_{t+1}}$ as a provisional $\mathbf{T}[s_i, t + 1]$ value and updates the priority of $(s_i, t + 1)$ accordingly. Thanks to the *monotonicity* of $\mathbf{T}[\cdot, \cdot]$ along a path, $\mathbf{T}[s_i, t]$ satisfies Equation (2) when (s_i, t) is popped from $\mathbf{Q}$. In the worst case, MINT visits each HMM edge once per time frame, hence takes $\mathcal{O}((K \log KT + |E|)T)$ time, where $\log KT$ expresses the priority queue overhead [Fredman and Tarjan, 1987]. In practice, it never considers some HMM edges. Next, we illustrate four MINT variants.

Standard MINT. In more detail, to achieve the $\max_{s_i} \mathbf{T}[s_i, T]$ objective, we *minimize* the positive path log-likelihood, $-\log P(Q, Y) \geq 0$, defined as *cost* of a path. For a given path Q ending at state i at frame t , we define its priority as $p(s_i, t) = -\log P(Q, Y)$, with $Q = \{s^1, s^2, \dots, s^t\}$, such that $s^t = s_i$ and $Y = \{y_1, y_2, \dots, y_t\}$. The pseudocode of the plain MINT variant is shown in Appendix D. First we insert in $\mathbf{Q}$ a token for each state in the first frame with priority $-\log \pi_{s_i} - \log B_{s_i, y_1}$. If a start state s is given, we only enqueue a token for s at $t = 0$. The main loop iterates

until reaching the last frame. In each iteration, we dequeue from $\mathbf{Q}$ the top token (s_h, t) , add it to a set $\mathbf{V}$ of visited tokens, compute the cost of reaching $(s_i, t + 1)$ via (s_h, t) for each out-neighbor s_i of s_h that has no such token in $\mathbf{V}$, and insert or update $(s_i, t + 1)$ in $\mathbf{Q}$ to the lowest known cost. When a pair (s, T) is dequeued, no path spanning T frames at lower cost exists, hence we return its cost as the Viterbi path log-likelihood $\max_{s_i} \mathbf{T}[s_i, T]$.

Proposition 1. *Standard MINT is correct.*

Proofs are in Appendix A. To return the optimal path, MINT by defaults appends a path to each token and, when updating the priority of $(s_i, t + 1)$, also updates the corresponding path to s_i . An alternative implementation, MINT-Backtracking, stores only the *predecessor* of each token, retains all explored tokens, and at the end constructs the optimal path by *backtracking*. Next, we present three extensions to MINT.

MINT Bound. We propose a variant of MINT that orders tokens by lower-bounding the path cost from each token (s_h, t) until the final frame T using a *lower bound* $\hat{c}_1$ on the cost for moving from one frame to another for the remaining $T - t$ frames. We call the ensuing algorithm MINT Bound.

We first insert all states (or the source state, if given) in $\mathbf{Q}$ with $p(s, 1) = (T - 1) \cdot \hat{c}_1$. As the search proceeds, we replace lower bounds with exact costs. The priority of (s_h, t) is $p(s_h, t) = -\log P(Q, Y) + (T - t) \cdot \hat{c}_1$, Q being the current optimal path ending at s_h in t frames; we compute the priority of a neighbour s_i for insertion or update in $\mathbf{Q}$ as $p(s_h, t) - \hat{c}_1 - \log A_{s_h, s_i} - \log B_{s_i, y_{t+1}}$. Upon reaching the last frame, all lower bounds capture exact costs. Setting $\hat{c}_1$ as the lowest value of $-\log A_{s_h, s_i} - \log B_{s_i, y_{t+1}}$ over all edges (s_h, s_i) at any t , found by pre-processing, ensures correctness.

Proposition 2. *MINT Bound is correct.*

MINT Bound encases information from unexamined time frames in the BestFS criterion and further explores already well-explored paths without compromising correctness.

Bidirectional MINT obtains more efficiency by bidirectional search [Pohl, 1969], exploring solutions both forward from the start and backward from the end time frame. We denote the graph in which all edges (i.e., neighbor relations) are reversed as G_{rev} . In each direction, the search proceeds as in Standard MINT, yet with two priority queues, $\mathbf{Q}_f$ (forward search) and $\mathbf{Q}_b$ (backward search). If an initial state is given and a final state is not given, we find, in pre-processing, all states $\mathcal{S}(T)$ reachable from the initial state in T frames, and initiate $\mathbf{Q}_b$ with those. The algorithm assumes that both an initial and final state are given. In each iteration, we expand both¹ searches, handling queues as in Standard MINT. Upon extracting (s_h^f, t^f) from $\mathbf{Q}_f$ and (s_h^b, t^b) from $\mathbf{Q}_b$, we update the associated visited-token sets, $\mathbf{V}_f$ and $\mathbf{V}_b$, and store associated costs in arrays d_f and d_b . We then consider tokens $(s_i^f, t^f + 1)$ for all neighbors s_i^f of s_h^f in G and tokens $(s_i^b, t^b - 1)$ for all neighbors s_i^b of s_h^b in the graph G_{rev} . We omit the details, which are the same as those in Standard MINT. When the two sides meet generating a path of length T , we update the

¹A more refined strategy would choose which side to expand, representing an opportunity for future work.

hitherto best path cost μ , if the newly found path improves on it. Such a path is provably optimal, hence the algorithm terminates, when the sum of costs of tokens dequeued from $\mathbf{Q}_f$ and $\mathbf{Q}_b$ exceeds μ . To avoid double-counting emission probabilities, we add the emission probability of the last vertex visited while building a path only in the priority of $\mathbf{Q}_f$, thus the cost of any path through (w, t) is $d_f[(w, t)] + d_b[(w, t)]$.

Proposition 3. *Bidirectional MINT is correct.*

Bidirectional MINT Bound combines Bidirectional MINT and MINT Bound in one algorithm that searches in both directions and lower-bounds the total T -frames-long path costs used as priority values in both queues. The search in both directions follows the order determined by the cost of arriving to a state in a given number of steps (frames) plus a lower bound on the cost of arriving from there to the end of the path. The single-frame lower bound for the forward search is as in MINT Bound. For backward search, it is the lowest cost of moving from a frame to the next one in the reverse HMM graph G_{rev} , i.e., the lowest value of $-\log A_{s_i, s_h} - \log B_{s_i, y_t}$ over all edges (s_i, s_h) in G_{rev} . We obtain both bounds by pre-processing with a single graph traversal. The correctness of Bidirectional MINT Bound follows from the correctness of MINT Bound and Bidirectional MINT.

Linear-space MINT (MINT-LS) Here, we propose a COMPACTBFS space-efficient variant, MINT-LS, which guarantees $\mathcal{O}(K)$ space by keeping the size of $\mathbf{Q}$ in $\mathcal{O}(K)$ by a *containment* strategy and reconstructs the solution by a *divide-and-conquer* strategy. Algorithm MINT-LS, summarized in Appendix D, invokes the DFS subroutine while storing, with each token (s_h, t) , its predecessor state and the running middle pair for its path. These details help identify the subproblems to be solved recursively. Upon reaching the final frame (and the final state, if specified), it extracts the middle pair associated with the solution path and reruns recursively among N_p -hop predecessors of the middle pair in as many preceding frames and among N_s -hop successors in as many following frames, identified through breadth-first search. MINT-LS avoids storing tokens when the queue size reaches a threshold θ . Instead, it invokes the DFS subroutine, which only stores tokens either as replacements of others or at the last frame. MINT-LS+ applies our EFS strategy that visits each token at most once prioritizing tokens by time frame with an auxiliary queue $\mathbf{Q}_t$; it keeps track of the earliest and latest time frame in $\mathbf{Q}_t$ and toggles between BestFS and EFS as required. As results in Section 4.1 show, these strategies ensure low space consumption. With large θ , DFS is preferable; otherwise, EFS is preferred. MINT-LS naturally combines with bidirectional search.

3.2 The TECH algorithm

We apply COMPACTBFS to V-Optimal histogram construction [Jagadish *et al.*, 1998], outlined in Section 2. A V-Optimal histogram uses the mean value $\hat{x} = \frac{1}{i-j+1} \sum_{k=j}^i x_k$ as a representative to minimize the *Euclidean* error in a bucket I_b extending from the j^{th} to the i^{th} value in the sequence, $E(j, i) = \sum_{k=j}^i (x_k - \hat{x})^2$. The total error is aggregated over all buckets. V-Optimal algorithms use incremental sums S and sums of squares SS to obtain any $E(j, i)$ as:

$$E(j, i) = (SS[i] - SS[j-1]) - \frac{(S[i] - S[j-1])^2}{(i-j+1)}. \quad (3)$$

The algorithm finds, for each combination (h, b) of a value index and number of buckets, the cost of the optimal b -bucket histogram covering the first i values in the sequence, as:

$$E^*(i, b) = \min_{1 \leq j < i} E^*(j, b-1) + E(j+1, i), \quad (4)$$

$E^*(i, b) = 0$ for $i \leq b$ and $E^*(i, 1) = E(1, i)$. After finding the optimal cost $E^*(n, B)$ for a length- n sequence and B buckets, we backtrack the histogram. The problem maps to COMPACTBFS as follows:

1. The *data space* $\mathcal{X}$ is the data series $I = \{x_1, \dots, x_n\}$.
2. The *neighbor* function $\mathcal{N}(x_i) = \{x_j | 1 \leq j < i\}$ generates a *set* $\mathbb{X}$ of *eligible sequences* (x_{i_t}) , $x_{i_t} \in \mathcal{X}$, $t \in \{1, \dots, B+1\}$ as *ordered* boundaries from x_1 to x_n .
3. The *gap function* $G(j, i, b)$ is the bucket error $E(j+1, i)$ irrespective of b .
4. The *selective dioid* is $(\mathbb{R} \cup \{+\infty\}, \min, +, +\infty, 0)$. Thus, the *value function* $f : \mathbb{X} \rightarrow \mathbb{R}$ assigns an approximation error $\sum_{b=1}^B E_b$ to a histogram.
5. The *problem* seeks an error-minimizing eligible histogram of $B+1$ boundaries ($\oplus := \min$).
6. The *recursive function* $\text{Opt}(i, \ell)$ that stores the *optimal* value for an eligible sequence of length ℓ ending at item $x_i \in \mathcal{X}$ is the function $E^*(i, b = \ell)$.
7. The *solution* by DP over sequences of increasing length from $\mathcal{X}$ is given by Equation (4).

V-OPT histogram construction by DP [Jagadish *et al.*, 1998] requires $\mathcal{O}(n^2 B)$ time and $\mathcal{O}(nB)$ space. We discuss COMPACTBFS variants, TECH (Time-Efficient Histogram), aligned to the variants of MINT.

Standard TECH. Standard TECH, like MINT, employs a priority queue $\mathbf{Q}$ to prune computations, where the priority of entry (h, b) is the cost of the b -bucket histogram for the first h values, $p(h, b) = E^*(h, b)$. After computing the S and SS arrays, used to compute the error measure by Equation (3), in each iteration, TECH dequeues the pair (h, b) of lowest error, adds it to a set $\mathbf{V}$ of visited tokens, and, provided $b < B$, computes via (h, b) the error for each pair $(i, b+1)$ with $i > h$ that is not in $\mathbf{V}$ and inserts or updates $(i, b+1)$ in $\mathbf{Q}$ accordingly; thereby, it explores possible next buckets. We do not iterate over $(i, b+1)$ pairs for all $i > h$, but stop at $i = n - B + b + 1$ since there must be at least $B - b - 1$ values after i to make B buckets in total. The algorithm terminates after it dequeues (n, B) . Correctness follows as in Proposition 1: after (n, B) is dequeued, there can be no lower-cost histogram of the same series and B . For further pruning, we use an upper bound $UB[h, b]$ on the cost of a $(B-b)$ -bucket histogram for the series $\{h+1, \dots, n\}$, derived in Proposition 4 below. If, after visiting (h, b) , we find $i^* > h$ such that $E(h+1, i^*) \geq UB[h, b]$, we eschew computing $E(h+1, i)$ for $i > i^*$, as we have already exceeded the upper bound on the error therefrom.

TECH Bound. This variant uses bounds on the cost of a B -bucket histogram instead of the cost of a partial histogram

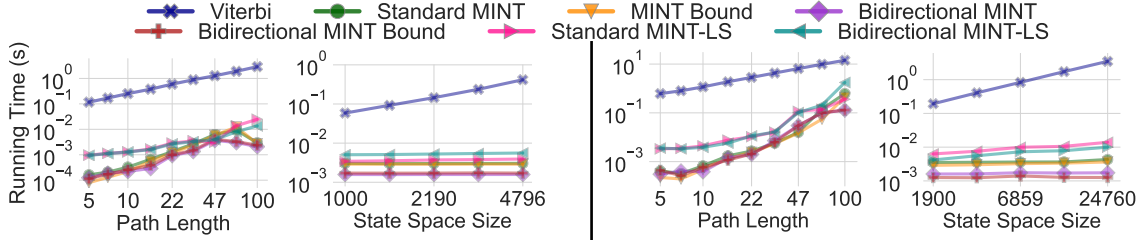


Figure 2: Runtime vs. T and K ; forced alignment (L), standard decoding (R); axes on log scale; real data.

with $b \leq B$ buckets. Given (h, b) , we partition the series $\{h+1, \dots, n\}$ in $B-b$ equal-width buckets. The minimum error among such buckets is a lower bound to the V-optimal histogram cost, while the sum of those errors is an upper bound, which we may use for pruning.

Proposition 4. *The minimum error $\min_b E_b$ among b buckets of an equal-width partitioning a sequence I is a lower bound to the V-optimal histogram cost and $\sum_b E_b$ is an upper bound.*

TECH Bound adds the lower bound $LB[h, b]$ to the priority of each pair (h, b) . Upon arrival at the end of the series, it outputs the same cost as Standard TECH. We find these bounds by building equal-width histograms in a pre-processing step. Correctness follows as in the case of MINT Bound.

Other TECH variants work by analogy to MINT variants. **Bidirectional TECH** applies forward and backward search. When we pop a pair (h, b) from Q_f , we consider entries $(i, b+1)$ with $i > h$, and, if the backward search has already visited $(i, B-b-1)$, we check whether the cost $d_f[(h, b)] + E(h, i) + d_b[(i, B-b-1)]$ improves upon the current best cost μ , and likewise in backward search. The search terminates when the sum of the costs of pairs from both queues exceeds the current best cost μ . **Bidirectional TECH Bound** combines Bidirectional TECH with TECH Bound, with reversed lower bounds to consider sequences of the form $\{1, \dots, h\}$ rather than $\{h+1, \dots, n\}$. **TECH-LS** variants limit space needs; in place of a *middle pair* of states, they detect a *middle bucket* that splits the data series in halves. Guha [2005] applied such a space-saving solution on DP.

4 Experiments

We experimented² on a 128GB, 2×12 core Xeon E5 2680 v3 machine @2.5GHz. For Viterbi decoding, we use as baselines the edge-aware Viterbi algorithm (§3.1), its recomputation-based space-efficient variant [Ciaperoni *et al.*, 2022], checkpoint Viterbi [Tarnas and Hughey, 1998], which segments a sequence into $\sqrt{T}$ parts, and SIEVE variants [Ciaperoni *et al.*, 2022], including SIEVE-Middlepath, Standard SIEVE (partitioning the state space), and SIEVE-Hyperloglog (using approximate counts [Flajolet *et al.*, 2007]). For histogram construction, the baseline is the DP algorithm of Jagadish *et al.* [1998] (§3.2). In Appendix B, we detail data, measures, and parameter choices, and Appendix C presents further experiments.

²Implementation and data available at <https://github.com/maciap/CompactBestFirst>.

4.1 Results on Viterbi decoding

Real data, runtime vs. T and K . Figure 2 plots results on forced-alignment and standard decoding with real data. Here, MINT achieves savings of up to three orders of magnitude over Viterbi by focusing on promising paths. Enlarging the state space via larger snowball samples of the HMM does not affect MINT, but heavily impacts Viterbi. MINT-LS variants also gain up to three orders of magnitude lower runtime than Viterbi even while controlling memory.

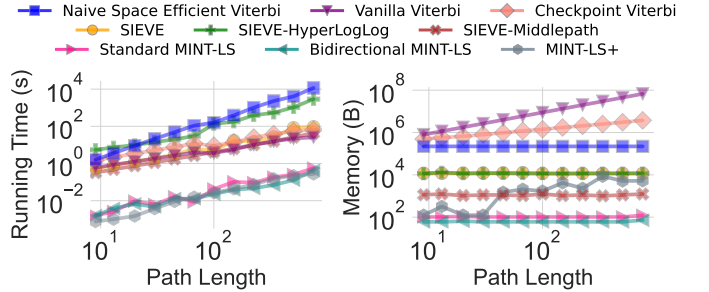


Figure 3: Runtime & (median over recursive calls) memory use vs T ; axes on log scale; real forced alignment data.

Real data, runtime and memory vs. T . Figure 3 plots runtime and memory usage, including SIEVE variants. As the needs of algorithms applying the COMPACTBFS strategy are *dynamic*, we report their median memory usage across recursion levels. The median memory needs of MINT-LS (using DFS) remain the lowest, while those of MINT-LS+ (using EFS) remain below those of the most memory-intensive SIEVE variants. Figure 11 in Appendix C additionally reports the corresponding maximum memory usage. Overall, MINT-LS variants require memory comparable to SIEVE variants. We show results vs. T , as subsampling the state space on real data barely affects MINT-LS, as established in Figure 2.

Effect of θ on linear-space algorithms. Figure 4 presents runtimes vs. the queue size threshold θ , focusing on ranges of the θ domain where MINT-LS may process tokens more than once, as explained in Section 3.1. MINT-LS+, on the other hand, never processes the same token more than once. SIEVE [Ciaperoni *et al.*, 2022], a DP-based rather than A*-based algorithm, ensures $\mathcal{O}(K)$ space complexity too, as it always stores K active tokens. However, SIEVE incurs a practical runtime overhead (*not time-complexity overhead*) compared to Viterbi; we thus compare runtime to the latter as a baseline. MINT-LS consistently performs the fastest on synthetic data with skewed path likelihood distribution, where only a few paths are explored, yet becomes inefficient for small θ with uniform path likelihood distribution, where more

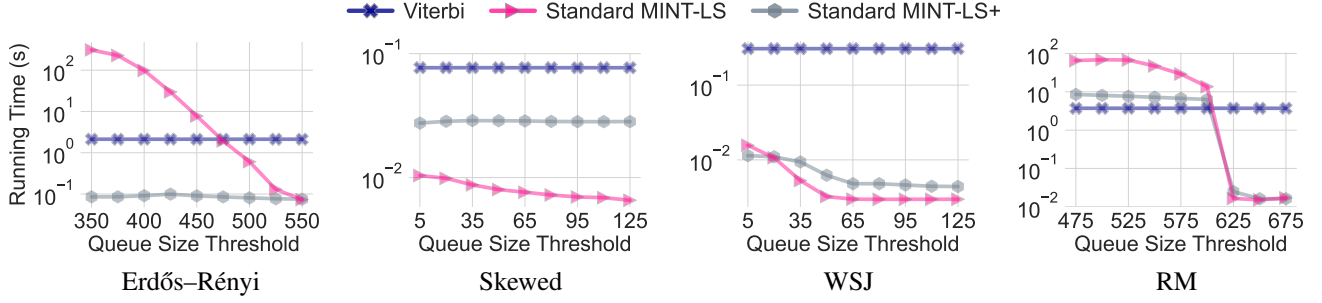


Figure 4: Runtime vs. queue size threshold θ ; synthetic data with uniform (Erdős-Rényi) and skewed path likelihood distribution (Skewed), and real data on forced alignment (WSJ) and standard decoding (RM); y -axis in log scale.

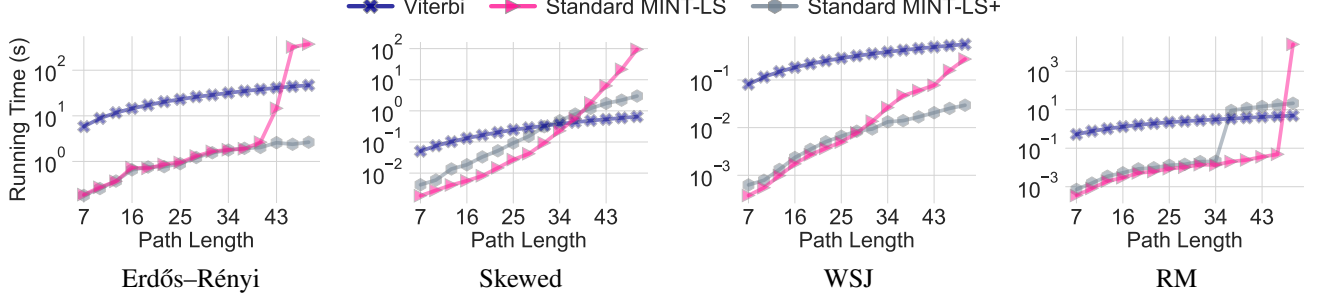


Figure 5: Runtime vs. path length T ; synthetic data with uniform (Erdős-Rényi) and skewed path likelihood distribution (Skewed), real data on forced alignment (WSJ) and standard decoding (RM); y -axis in log scale.

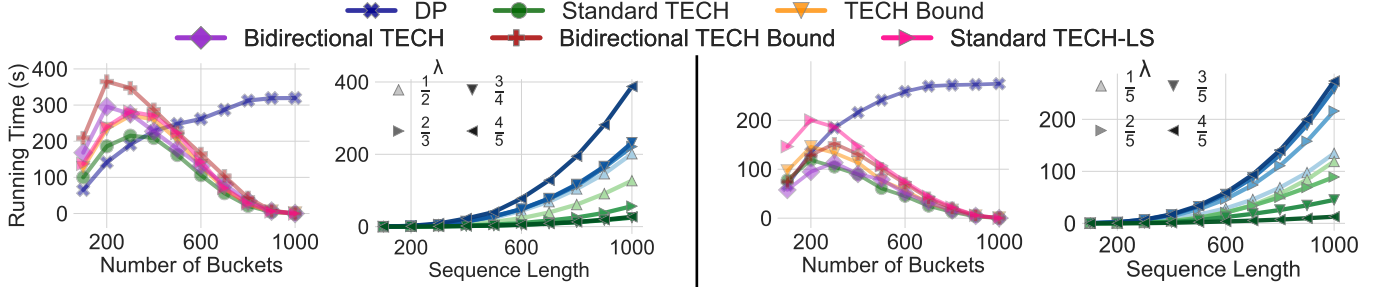


Figure 6: Histogram construction: runtime vs. B , synthetic (L) and real (R) data; n varying $\lambda = \frac{B}{n}$.

paths are explored. On real data with forced alignment, the runtime of MINT-LS grows modestly as θ falls; with larger decoding data, runtime grows beyond that of Viterbi. Overall, MINT-LS is preferable for large enough θ , while MINT-LS+ maintains runtime within the same order of magnitude as, and typically lower than, that of Viterbi.

Effect of T on linear-space algorithms. Figure 5 plots the runtimes vs. path length T for fixed values of the queue size threshold θ sufficiently small to reveal the challenges MINT-LS faces. The results reconfirm that MINT-LS becomes inefficient as more paths are explored. As in Figure 4, this effect is evident in synthetic data with Erdős-Rényi transitions and uniform path likelihood distribution and in the real decoding data. MINT-LS+ emerges as the algorithm of choice, achieving competitive runtime even while maintaining memory usage under a small θ threshold.

4.2 Results on histogram construction

Figure 6 shows the histogram construction runtime on synthetic data vs. B for input sequence length $n = 1010$, and that of Standard TECH and the standard V-OPT algorithm vs. n

for different values of $\lambda = \frac{B}{n}$. Standard TECH often suffices, as its variants do not bring significant gains. Linear-space TECH incurs a manageable runtime overhead for the sake of space efficiency. All COMPACTBFS variants gain in time efficiency as B grows, delimiting the search space for each bucket. Contrariwise, standard DP cannot contain the search space, hence its runtime surges for large B . Figure 6 also shows results on real data. The gains of TECH solutions are *more emphatic* here, as TECH exploits data patterns that facilitate summarization, whereas standard DP lacks such capacity.

5 Conclusion

We introduced COMPACTBFS, a framework that efficiently solves fixed-length path optimization problems by best-first search, while delimiting space complexity. We crafted COMPACTBFS-based algorithms for Viterbi decoding and histogram construction. As our experiments evince, COMPACTBFS gains up to four orders of magnitude in time and space efficiency, the gain being more pronounced on real data with nonuniform path cost distributions. In Appendix E, we apply COMPACTBFS to temporal-graph community search.

References

- Arturs Backurs and Christos Tzamos. Improving Viterbi is hard: Better runtimes imply faster clique algorithms. In *ICML*, pages 311–321, 2017.
- Richard Bellman. Dynamic programming. *Science*, 153(3731):34–37, 1966.
- Amaury Van Bemten, Jochen W. Guck, Carmen Mas Machuca, and Wolfgang Kellerer. Bounded Dijkstra (BD): Search space reduction for expediting shortest path subroutines. *CoRR*, abs/1903.00436, 2019.
- John Binder, Kevin P. Murphy, and Stuart Russell. Space-efficient inference in dynamic probabilistic networks. In *IJCAI*, pages 1292–1296, 1997.
- Hugo Braun, Justin Luitjens, Ryan Leary, Tim Kaldewey, and Daniel Povey. Gpu-accelerated viterbi exact lattice decoder for batched online and offline speech recognition. In *ICASSP*, pages 7874–7878, 2020.
- Martino Ciaperoni, Aristides Gionis, Athanasios Katsamanis, and Panagiotis Karras. SIEVE: A space-efficient algorithm for Viterbi decoding. In *ACM SIGMOD*, pages 1136–1145, 2022.
- Martino Ciaperoni, Athanasios Katsamanis, Aristides Gionis, and Panagiotis Karras. Beam-search SIEVE for low-memory speech recognition. In *Interspeech 2024*, pages 272–276, 2024.
- Edsger W Dijkstra et al. A note on two problems in connexion with graphs. *Numerische mathematik*, 1(1):269–271, 1959.
- Pedro F. Felzenszwalb, Daniel P. Huttenlocher, and Jon M. Kleinberg. Fast algorithms for large-state-space HMMs with applications to web usage analysis. In *NeurIPS*, pages 409–416, 2003.
- Philippe Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier. HyperLogLog: the analysis of a near-optimal cardinality estimation algorithm. In *Conference on Analysis of Algorithms (AofA)*, pages 137–156, 2007.
- Daniel Foead, Alifio Ghifari, Marchel Budi Kusuma, Novita Hanafiah, and Eric Gunawan. A systematic literature review of A* pathfinding. *Procedia Computer Science*, 179:507–514, 2021.
- Michael L. Fredman and Robert Endre Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. *J. ACM*, 34(3):596–615, 1987.
- Mark J. F. Gales and Steve J. Young. The application of hidden markov models in speech recognition. *Foundations and Trends in Signal Processing*, 1(3):195–304, 2007.
- Edoardo Galimberti, Martino Ciaperoni, Alain Barrat, Francesco Bonchi, Ciro Cattuto, and Francesco Gullo. Spancore decomposition for temporal networks: Algorithms and applications. *ACM Trans. Knowl. Discov. Data*, 15(1):2:1–2:44, 2021.
- John S. Garofolo, Lori F. Lamel, William M. Fisher, Jonathan G. Fiscus, David S. Pallett, and Nancy L. Dahlgren. *DARPA TIMIT acoustic-phonetic continuous speech corpus CD-ROM. NIST speech disc 1-1.1*. National Institute of Standards and Technology, 1993.
- Michel Gondran and Michel Minoux. *Graphs, Dioids and Semirings: New Models and Algorithms (Operations Research/Computer Science Interfaces Series)*. Springer, 2008.
- Sudipto Guha, Nick Koudas, and Kyuseok Shim. Approximation and streaming algorithms for histogram construction problems. *ACM Trans. Database Syst.*, 31(1):396–438, 2006.
- Sudipto Guha. Space efficiency in synopsis construction algorithms. In *VLDB*, pages 409–420, 2005.
- Felix Halim, Panagiotis Karras, and Roland HC Yap. Fast and effective histogram construction. In *CIKM*, pages 1167–1176, 2009.
- Daniel S. Hirschberg. A linear space algorithm for computing maximal common subsequences. *Commun. ACM*, 18(6):341–343, 1975.
- Daniel S. Hirschberg. Algorithms for the longest common subsequence problem. *J. ACM*, 24(4):664–675, 1977.
- Zhiheng Huang, Yi Chang, Bo Long, Jean-François Crespo, Anlei Dong, S. Sathya Keerthi, and Su-Lin Wu. Iterative Viterbi A* algorithm for k -best sequential decoding. In *ACL*, pages 611–619, 2012.
- Liang Huang. Advanced dynamic programming in semiring and hypergraph frameworks. In *Coling 2008: Advanced Dynamic Programming in Computational Linguistics: Theory, Algorithms and Applications - Tutorial notes*, pages 1–18, 2008.
- Hosagrahar Visvesvaraya Jagadish, Nick Koudas, Shanmugavelayutham Muthukrishnan, Viswanath Poosala, Kenneth Clem Sevcik, and Torsten Suel. Optimal histograms with quality guarantees. In *VLDB*, pages 275–286, 1998.
- Jihyuk Jo, Han-Gyu Kim, In-Cheol Park, Bang Chul Jung, and Hoyoung Yoo. Modified viterbi scoring for hmm-based speech recognition. *Intelligent Automation & Soft Computing*, 25(2):351–358, 2019.
- Panagiotis Karras and Nikos Mamoulis. Hierarchical synopses with optimal error guarantees. *ACM Transaction on Database Systems*, 33(3):18:1–18:53, 2008.
- Dan Klein and Christopher D. Manning. A* parsing: Fast exact Viterbi parse selection. In *HLT-NAACL*, pages 119–126, 2003.
- Richard E. Korf and Weixiong Zhang. Divide-and-conquer frontier search applied to optimal sequence alignment. In *AAAI*, pages 910–916, 2000.
- Richard E. Korf. A divide and conquer bidirectional search: First results. In *IJCAI*, pages 1184–1191, 1999.
- Rossana Mastrandrea, Julie Fournet, and Alain Barrat. Contact patterns in a high school: a comparison between data collected using wearable sensors, contact diaries and friendship surveys. *PloS one*, 10(9):e0136497, 2015.

- Mehryar Mohri. Semiring frameworks and algorithms for shortest-distance problems. *J. Autom. Lang. Comb.*, 7(3):321–350, 2002.
- Douglas B. Paul and Janet M. Baker. The design for the wall street journal-based CSR corpus. In *The 2nd International Conference on Spoken Language Processing (ICSLP)*, 1992.
- Judea Pearl. *Heuristics: intelligent search strategies for computer problem solving*. Addison-Wesley Longman Publishing Co., Inc., 1984.
- Ira Pohl. Bi-directional and heuristic search in path problems. Technical report, Stanford Linear Accelerator Center, Calif., 1969.
- Daniel Povey, Arnab Ghoshal, Gilles Boulianne, Lukas Burget, Ondrej Glembek, Nagendra Goel, Mirko Hannemann, Petr Motlicek, Yanmin Qian, Petr Schwarz, Jan Silovsky, Georg Stemmer, and Karel Vesely. The kaldi speech recognition toolkit. In *IEEE Workshop on Automatic Speech Recognition and Understanding (ASRU)*, 2011.
- Vineel Pratap, Andros Tjandra, Bowen Shi, Paden Tomasello, Arun Babu, Sayani Kundu, Ali Elkahky, Zhaoeng Ni, Apoorv Vyas, Maryam Fazel-Zarandi, Alexei Baevski, Yossi Adi, Xiaohui Zhang, Wei-Ning Hsu, Alexis Conneau, and Michael Auli. Scaling speech technology to 1, 000+ languages. *J. Mach. Learn. Res.*, 25:97:1–97:52, 2024.
- P. Price, W. M. Fisher, Jared Bernstein, and D. S. Pallett. Resource management rm2 2.0. *DVD. Philadelphia: Linguistic Data Consortium*, 1993.
- Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Prentice Hall, 3 edition, 2010.
- Sajid M. Siddiqi and Andrew W. Moore. Fast inference and learning in large-state-space HMMs. In *Proceedings of the 22nd International Conference on Machine learning (ICML)*, pages 800–807, 2005.
- Moshe Sniedovich. Dijkstra’s algorithm revisited: the dynamic programming connexion. *Control and cybernetics*, 35(3):599–620, 2006.
- John L. Spouge. Speeding up dynamic programming algorithms for finding optimal lattice paths. *SIAM Journal on Applied Mathematics*, 49(5):1552–1566, 1989.
- John L. Spouge. Fast optimal alignment. *Comput. Appl. Biosci.*, 7(1):1–7, 1991.
- Christopher Tarnas and Richard Hughey. Reduced space hidden Markov model training. *Bioinformatics*, 14(5):401–406, 1998.
- Nikolaos Tziavelis, Wolfgang Gatterbauer, and Mirek Riedewald. Any-k algorithms for enumerating ranked answers to conjunctive queries. *ACM Trans. Database Syst.*, 51(1), September 2025.
- Andrew J. Viterbi. Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory*, 13(2):260–269, 1967.
- Andrew J. Viterbi. A personal history of the viterbi algorithm. *IEEE Signal Processing Magazine*, 23(4):120–142, 2006.
- Steve J. Young, N.H. Russell, and J.H.S. Thornton. *Token passing: a simple conceptual model for connected speech recognition systems*. University of Cambridge, Department of Engineering, 1989.
- Steve J. Young, Gunnar Evermann, Mark J. F. Gales, Thomas Hain, Dan Kershaw, Xunying Liu, Gareth Moore, Julian Odell, Dave Ollason, Dan Povey, et al. *The HTK Book*. University of Cambridge, Department of Engineering, 2002.

A Proofs

This section collects proofs.

Proposition 1. *Standard MINT is correct.*

Proof. Let $\mathcal{T}$ be the set of all tokens (i.e., state-frame pairs) and $\mathbf{V}$ the set of visited tokens, for which optimal cost has been computed. Initially $\mathbf{V}$ includes the source s with optimal cost $p_s = -\log B_{s,y_1}$, and $-p_s$ is its log-likelihood. In each iteration, we add to $\mathbf{V}$ a token (s_h, t) from $\mathcal{T} \setminus \mathbf{V}$ with cost p_i . To complete the induction, we must show that p_i is optimal for its length t . First, if the true t -length-optimal path goes only through tokens in $\mathbf{V}$, for which, by the inductive hypothesis, the optimal cost is known, then p_i must be t -length-optimal. Assume that the t -length-optimal path goes through a token (s_j, t') not in $\mathbf{V}$; this token necessarily has cost $p_j \geq p_i$, hence any path through it is suboptimal. Therefore, $-p_i$ is the maximum Viterbi path log-likelihood $\max_{s_i} \mathbf{T}[s_i, t]$. $\square$

Proposition 2. *MINT Bound is correct.*

Proof. We prove the statement by contradiction. Assume the algorithm returns a token (s^*, T) with cost $c^* > \max_{s_i} \mathbf{T}[s_i, T]$. Then there must be an unvisited token (s', t') , which, if propagated to the last frame, produces a path of likelihood $\max_{s_i} \mathbf{T}[s_i, T]$. Such a token would have priority $c' + (T - t') \cdot \hat{c}_1$ where c' is the real cost of arriving to s' in t' frames. Since $\hat{c}_1$ lower-bounds the cost of moving from frame to frame, it follows that $c' + (T - t') \cdot \hat{c}_1 \leq \max_{s_i} \mathbf{T}[s_i, T] < c^*$ at any time $t' \leq T$. Therefore (s^*, T) cannot be dequeued before (s', t') , ergo the proof is completed. $\square$

Proposition 3. *Bidirectional MINT is correct.*

Proof. The algorithm alternates between a forward and backward step and maintains the best-so-far path of T steps. Correctness rests on the stopping condition. The algorithm terminates when either (i) both queues are empty or (ii) the elements (s_i^f, t^f) and (s_i^b, t^b) popped from the queues have joint cost $d_f[(s_i^f, t^f)] + d_b[(s_i^b, t^b)]$ that is larger than the current best path cost μ . Regarding condition (i), when both queues are empty, all possible paths have been generated, so the algorithm returns the optimal. Regarding condition (ii), assume that the optimal path is not yet found when the algorithm terminates returning $-\mu$. Then there must be a path Q^* of cost $\mu^* < \mu$ containing at least one not yet visited token in $\mathbf{V}_f \cup \mathbf{V}_b$. Such a token would have cost at least $d_f[(s_i^f, t^f)]$ on the forward side and $d_b[(s_i^b, t^b)]$ on the backward side, hence path Q^* would have cost $\mu^* \geq \mu$, a contradiction. $\square$

Proposition 4. *The minimum error $\min_b E_b$ among b buckets of an equal-width partitioning a sequence I is a lower bound to the V-optimal histogram cost and $\sum_b E_b$ is an upper bound.*

Proof. Let H_B be the V-optimal histogram of size B on sequence I and H'_B be the histogram of the same number of buckets on the same sequence, where all buckets have the same size, except possibly the last. Let $[s_j, e_j]$ be the boundary positions and E_j the error of the j^{th} bucket in H_B , and let $[s'_j, e'_j]$ and E'_j be the corresponding boundary positions and error of the j^{th} bucket in H'_B . Then at least one bucket

of H_B , say the j^{th} , has $a_j \leq a'_j$ and $b_j \geq b'_j$, i.e., it fully contains the corresponding bucket in H'_B . Then, as E is monotonically non-decreasing with bucket width, $E_j \geq E'_j$; besides, $E_j \leq \sum_{h \in H_B} E_h$, ergo $\min_h E'_h \leq E'_j \leq E_j \leq \sum_{h \in H_B} E_h$, hence $\min_h E'_h$ is a lower bound to the V-optimal histogram cost. Furthermore, $\sum_h E'_h$ is an upper bound on the V-OPT histogram cost, since, by definition of the V-OPT histogram H_B , $\sum_h E_h \leq \sum_h E'_h$. $\square$

B Data, Measures, and Parameters

This section provides details on the experimental setup, including the data, evaluation measures, and parameter settings.

Data. We experiment on both synthetic and real-world datasets.

We evaluate MINT and TECH variants on synthetic data generated according to the following models:

- *Erdős-Rényi* model where each hidden state is emitting and connected with any other state with probability $p = 0.01$; transition and emission probabilities are generated uniformly at random, thus arbitrary cycles may be present. All states are emitting.
- *Skewed path likelihood* model, where we generate a fixed number N_{path} (100, by default) of paths of length T starting from initial state s , and composed of emitting states. To each such path we assign a probability drawn from a power law distribution $p(x, \alpha) = \alpha x^{\alpha-1}$, which we distribute evenly across transition and emission probabilities of all states in the path. As in the previous case, all states are emitting. We use this model to investigate how skew in the distribution of path likelihoods affects the advantage granted by COMPACTBFS-based solutions.

We also assess MINT on real speech data:

- *Wall Street Journal (WSJ) corpus* data: we use a real-world composite HMM for speech-text forced alignment, the process of aligning text to audio recordings, which is also tackled by the Viterbi algorithm. The model is built using the HTK software toolkit Young *et al.* [2002] and contains 5529 states (including initial states), out of which 3204 are emitting; it was trained on the WSJ corpus Paul and Baker [1992] aiming to align speech recordings from the TIMIT corpus Garofolo *et al.* [1993].
- *Resource Management (RM) corpus* data: we use a real-world composite HMM for decoding, trained on the RM speech corpus Price *et al.* [1993] and built using the Kaldi software toolkit Povey *et al.* [2011]. The graph comprises 25 333 states (including initial state) and 175 428 edges, out of which 162 255 also carry emission probabilities. We decode subsets of a simple recorded utterance of up to 100 frames.

The observation sequence Y consists of feature vectors of Mel-Cepstrum cepstral coefficients and their derivatives and emission probabilities are given by multivariate Gaussian mixture models.

Similarly, we evaluate TECH in (i) synthetic *sequences of integers* in the range $[0, 50]$ and (ii) *Dow-Jones Industrial Average (DJIA) closing values* real data.

Metrics. We measure runtime in seconds (s) and memory in bytes (B). In all cases, we report averages over 5 runs.

Parameters. Regarding decoding in HMMs, in experiments with synthetic data, we vary T in geometric progression of 9 values from 5 to 30 with $K = 7500$; in experiments with real data, we vary T in geometric progression of 9 values from 5 to 100 with K fixed to the size of the original state space. Furthermore, in synthetic data we vary K in a geometric progression of 5 values from 1000 to 16000 with $T = 10$; in real data, we vary K over 5 values in geometric progression from 1000 (forced alignment) or 1900 (decoding) to approximately the size of the original state space, with $T = 30$. To vary K in real-world HMMs, we sample subsets of the original HMM graph via snowball sampling from a start state *source*. To investigate the combined impact of K and T , we also vary them simultaneously. We also vary $K \in \{25 \times 10^3, 30 \times 10^3\}$ and $T \in \{25, 30\}$ to monitor how MINT variants behave during runtime in terms of memory usage and the evolution of path likelihood. In the experiment with the skewed path likelihood model, we also vary the power law parameter α controlling skewness in 17 values from 10^{-2} to 10^2 with $N_{path} = 100$, $T = 10$ and $K = 1500$. In the space-efficient standard and bidirectional MINT-LS, unless specified otherwise, we set the queue size threshold θ to 10% of $K \times T$. However, with Erdős–Rényi data, which call for a larger budget as they reflect a worst-case scenario, we set θ to 90% of $K \times T$. In MINT-LS+, unless specified otherwise, we set $\theta = K$.

To assess the runtime of MINT-LS to MINT-LS+ as a function of the queue size threshold θ , we use synthetic data with $K = 7500$ and $T = 10$ and real data with K defined by the state space and $T = 30$, varying θ in arithmetic progression with step 15 from 5 to 125 on synthetic data with skewed path likelihood distribution and real forced alignment data, where the amounts of paths to be explored is limited, from 350 to 550 on Erdős–Rényi data, and from 475 to 675 on real decoding data. We set the Δ_Q in MINT-LS+ to 2. To assess the runtime of MINT-LS to MINT-LS+ as a function of path length T , we use synthetic data with $K = 16000$ and real data with K defined by the state space, varying T in arithmetic progression with step 3 from 7 to 49. We set $\theta = 2700$ on synthetic Erdős–Rényi data, $\theta = 10$ on synthetic data with skewed path likelihood distribution, $\theta = 20$ on real forced alignment data, and $\theta = 800$ on real decoding data, and Δ_Q in MINT-LS+ to 5.

In histogram construction experiments, we vary B from 100 to 1000, while holding n fixed to 1010. We also consider sequences of length n increasing from 100 to 1000 for the different values of $\lambda = \frac{B}{n}$ indicated in the results. With TECH-LS, we set the queue size threshold to 10% of $n \times B$.

C Additional Experiments

This section presents additional experiments that further support and extend the findings reported in Section 4.

Synthetic data, runtime vs. T and K . Figure 7 plots runtime vs. path length T and state space size K on Erdős–Rényi data. MINT accelerates decoding significantly, thanks to visiting

only a few tokens. MINT-LS incurs marginal runtime overhead compared to MINT. Still, processing a single token is faster in Viterbi, as MINT’s queue management incurs overhead, causing savings to drop as T grows. This result stems from the data model, which reflects a worst-case scenario whereby path likelihoods converge for large enough T , leading MINT to visit too many tokens. Conversely, in real-world speech data with probabilities concentrated over a limited subset of paths, MINT yields higher savings by focusing on the most promising paths.

Runtime vs. α . To demonstrate the effect of path likelihood skew using synthetic data, Figure 8 plots runtime as a function of the parameter α of the power law distribution over the path likelihoods. Notably, the highest savings are obtained for α close to 1. This is due to the fact that, for remarkably smaller or larger α , all paths tend to be equally likely, so MINT cannot focus on a small subset of paths. Nevertheless, even in the case where the path likelihood distribution approaches the worst case, as in the Erdős–Rényi model, we still have high savings for small T , which is a popular setting in modern speech recognition. Regarding different implementations of MINT, we observe that the use of lower bounds is not always crucial for runtime; however, as α increases, MINT Bound vastly outperforms Standard MINT by virtue of its capacity to prune paths from consideration more aggressively. Furthermore, bidirectional-search variants accomplish the highest efficiency on synthetic data, both those generated by the Erdős–Rényi model and those with skewed path likelihoods; these results vindicate our development of those enhanced solutions.

Runtime vs. T and K tuned in unison. We also measure runtime as a function of both T and K on Erdős–Rényi model data and on those with skewed path likelihood distribution with $\alpha = 1$. Figure 8 shows the results. Shaded regions indicate the minimum and the maximum over runs, which convey the extent of random variation; as the figure shows, that extent is quite limited. The savings observed as we increase both T and K are consistent with our previous findings and most pronounced in the skewed likelihoods scenario. In the Erdős–Rényi model, as most paths of a given length have similar likelihoods, the savings are more modest and decrease with the growth of both T and K .

Real-time memory monitoring. Figure 9 shows memory requirements at run time for four parameter configurations on synthetic data with skewed path likelihood distribution using $\alpha = 1$; for reference, we also provide the constant memory used by standard Viterbi. Notably, MINT variants reduce the memory requirements of Viterbi by several orders of magnitude. Unsurprisingly, the two bidirectional-search variants consume slightly more memory, yet need fewer iterations till termination, as they apply both a forward and a backward search with two queues. With MINT-LS, we show memory consumption vs. iterations or DFS calls. While by the chosen budget, MINT-LS variants use as little as 25% of the memory used by MINT variants; this advantage may grow on demand by reducing the queue size threshold θ .

Effect of backtracking. As explained in Section 3.1, by default MINT variants store paths explicitly; however, we may reduce memory usage by only storing the predecessor of each

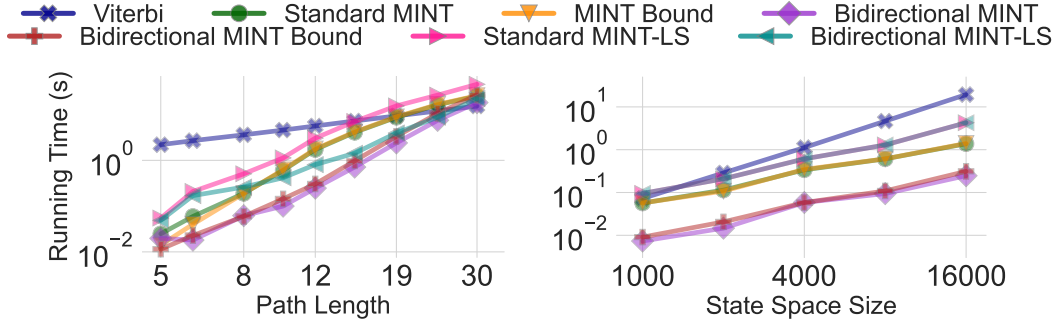


Figure 7: Decoding, synthetic data. Runtime vs. path length (left) and state space size (right).

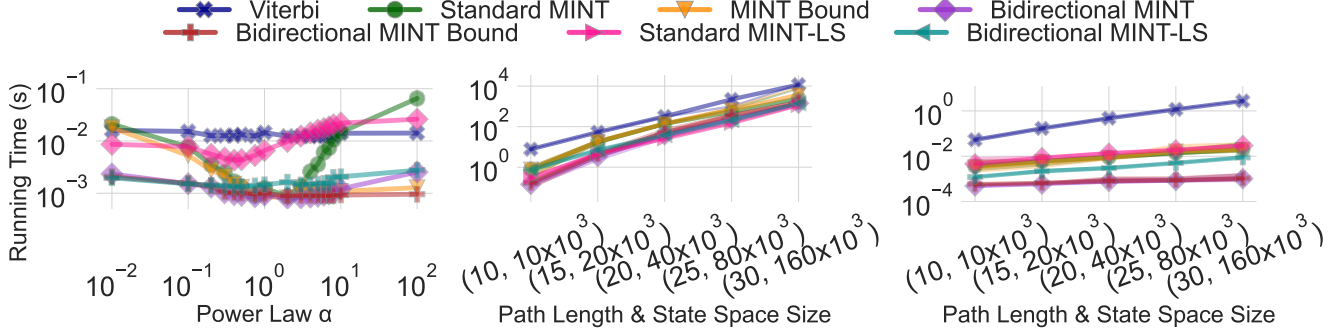


Figure 8: Decoding; synthetic data with skewed path likelihoods; runtime vs. skew α (left); synthetic Erdős-Rényi data (center) and synthetic data with skewed path likelihoods (right): runtime vs. both linearly growing path length T and exponentially growing state space size K , indicated as (T, K) ; shaded regions indicate the minimum and maximum runtime due to randomness in data generation; both axes on log scale (left) and y -axis in log scale (center and right).

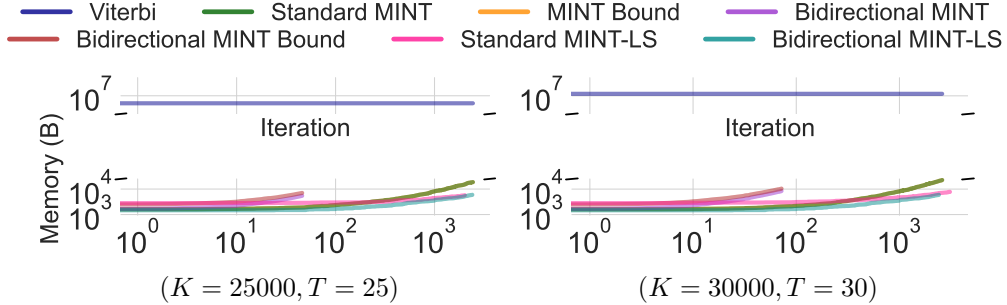


Figure 9: Synthetic data with skewed path likelihood distribution; memory requirements on the fly.

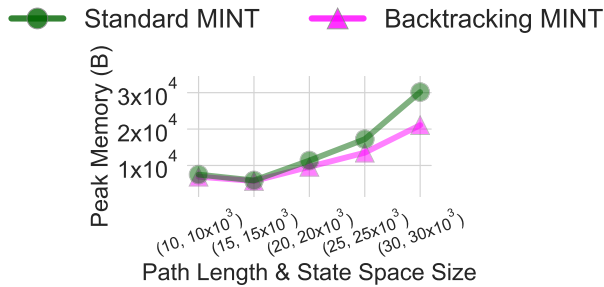


Figure 10: Decoding, synthetic data with skewed path likelihood distribution. Memory requirements of default MINT and memory-efficient MINT-Backtracking. Maximum (peak) memory vs. path length T and state space size K , indicated as (T, K) ; $\frac{K}{T}$ is fixed.

token (s_i, t) and eventually reconstructing the optimal path by backtracking over such links, with a small runtime over-

head and savings in memory consumption. We refer to the resulting implementation as MINT-Backtracking. To illustrate this effect, Figure 10 presents the maximum memory usage of the two implementations of standard MINT under the HMM graph model with skewed path likelihood distribution ($\alpha = 1$) as a function of both K and T . While the difference in memory requirement is evident, we measured the corresponding runtime difference to be negligible. MINT-LS employs the backtracking implementation for the sake of space efficiency. Besides, MINT-Backtracking extends seamlessly to all variants of MINT. In the case of bidirectional-search-based variants, backtracking proceeds in both directions after the optimal path is found.

Maximum memory over recursive calls. The memory requirements of the SIEVE variants and the space-efficient COMPACTBFS-based algorithms vary across recursive calls. While

Figure 3 in Section 4.1 reports median memory usage over recursive calls, Figure 11 reports maximum, i.e., peak, memory usage, including the same baselines with static memory requirements.

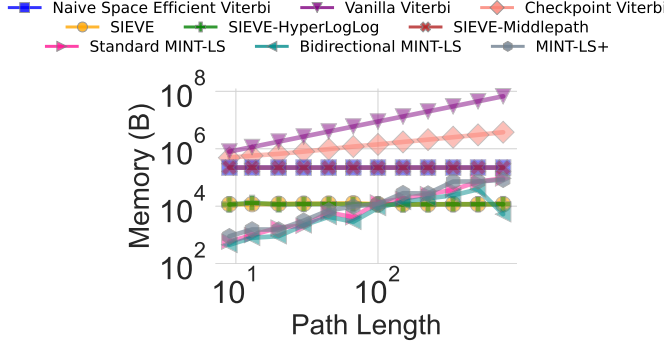


Figure 11: Peak memory use (over recursive calls) vs T ; axes on log scale; real forced alignment data.

Real-time log-likelihood monitoring. We also monitor the optimal path log-likelihood absolute value across iterations. This *absolute* value grows, as longer paths have *lower* likelihood than shorter ones. Figure 12 presents our results, using the same four parameter configurations as in Figure 9. In all algorithms the likelihood approaches the optimal value swiftly and monotonically. In the case of standard Viterbi, we plot the highest likelihood found at the end of each frame (i.e., path length considered), hence Viterbi appears to undergo fewer iterations. For the two bidirectional MINT variants, we plot the sum of likelihoods associated with the last tokens dequeued from the forward queue Q_f and the backward queue Q_b in each iteration. For other algorithms, we plot the likelihood associated with the token dequeued from Q in each iteration. We found that MINT-LS variants exhibit the same progression of path likelihood as the corresponding MINT variants.

Algorithm 1: Standard MINT

Data: HMM graph G , transition and emission probabilities A and B , observations Y , and initial state s .
Result: Viterbi Path Log-Likelihood $\max_{s_h} \mathbf{T}[s_h, T]$.

```

1  $Q \leftarrow Queue((s, 1), p(s, 1) = -\log B_{s, y_1});$ 
2  $V \leftarrow \{\};$ 
3 while  $Q \neq \emptyset$  do
4    $(s_h, t), p \leftarrow Q.pop();$  // (state, frame), priority
5   if  $t = T$  then break;
6    $V.add((s_h, t));$ 
7   for  $s_i \in \mathcal{N}_{out}(s_h)$  do
8     if  $(s_i, t+1) \notin V$  then
9        $d \leftarrow p - \log A_{s_h, s_i} - \log B_{s_i, y_{t+1}};$ 
10      if  $(s_i, t+1) \notin Q$  then
11         $Q.insert((s_i, t+1), p(s_i, t+1) = d);$ 
12      if  $Q[(s_i, t+1)] > d$  then
13         $Q.update((s_i, t+1), p(s_i, t+1) = d);$ 
14 return  $-p;$ 
```

D Pseudocodes

Here we collect pseudocodes for the presented algorithms.

Algorithm 1 presents MINT. Algorithm 2 presents Bidirectional MINT, assuming that both an initial and final state are given. Algorithm 3 illustrates TECH. Algorithm 5 provides the pseudocode of MINT-LS, which uses Algorithm 4 as a subroutine. Lastly, Algorithm 6 presents MINT-LS+.

Algorithm 2: Bidirectional MINT

Data: HMM graph G , transition and emission probabilities A and B , observations Y , and initial and final states $source$ and $target$.
Result: Viterbi Path Log-Likelihood $\max_{s_h} \mathbf{T}[s_h, T]$.

```

1  $Q_f \leftarrow Queue((source, 1), p(source, 1) = -\log B_{source, y_1});$ 
2  $Q_b \leftarrow Queue((target, T), p(target, T) = 0);$ 
3  $V_f \leftarrow \{\}; V_b \leftarrow \{\}; \mu \leftarrow \infty;$ 
4 while  $Q_b \neq \emptyset \wedge Q_f \neq \emptyset$  do
5    $(s_h^f, t^f), p_h^f \leftarrow Q_f.pop(); (s_h^b, t^b), p_h^b \leftarrow Q_b.pop();$ 
6    $d_f[s_h^f] \leftarrow p_h^f; d_b[s_h^b] \leftarrow p_h^b;$ 
7    $V_f.add((s_h^f, t^f)); V_b.add((s_h^b, t^b));$ 
8   if  $t^f < T$  then
9     for  $s_i \in \mathcal{N}_{out}(s_h^f)$  do
10       Update  $Q_f$  for  $(s_i, t+1);$ 
11        $d = d_f[(s_h^f, t^f)] - \log A_{s_h^f, s_i} - \log B_{s_i, y_{t+1}} + d_b[(s_i, t+1)];$ 
12       if  $(s_i, t+1) \in V_b \wedge d < \mu$  then
13          $\mu = d;$ 
14   if  $t^b > 1$  then
15     for  $s_i \in \mathcal{N}_{in}(s_h^b)$  do
16       Update  $Q_b$  for  $(s_i, t-1);$ 
17        $d = d_b[(s_h^b, t^b)] - \log A_{s_i, s_h^b} - \log B_{s_h^b, y_t} + d_f[(s_i, t-1)];$ 
18       if  $(s_i, t-1) \in V_f \wedge d < \mu$  then
19          $\mu = d;$ 
20   if  $d_f[(s_h^f, t^f)] + d_b[(s_h^b, t^b)] \geq \mu$  then break;
21 return  $-\mu;$ 
```

Algorithm 3: Standard TECH

Data: Input sequence I , integer B .
Result: V-Optimal Histogram Error $E^*(n, B)$.

```

1  $Q \leftarrow Queue((1, 1), p(1, 1) = 0);$ 
2  $S \leftarrow \{\}; SS \leftarrow \{\}; V \leftarrow \{\}; n \leftarrow I.length;$ 
3  $S[1] \leftarrow I[1]; SS[1] \leftarrow I[1]^2;$ 
4 for  $i \in \{2, \dots, n\}$  do
5    $S[i] \leftarrow S[i-1] + I[i]; SS[i] \leftarrow SS[i-1] + I[i]^2;$ 
6 for  $i \in \{2, \dots, n - (B-1)\}$  do
7    $Q.insert((i, 1), p(i, 1) = E(1, i));$ 
8 while  $Q \neq \emptyset$  do
9    $(h, b), p \leftarrow Q.pop();$  // (values, buckets), priority
10  if  $b = B \wedge h = n$  then break;
11   $V.add((h, b));$ 
12  if  $b < B$  then
13    for  $i \in \{h+1, \dots, n - B + b + 1\}$  do
14      if  $(i, b+1) \notin V$  then
15         $d \leftarrow p + E(h+1, i);$ 
16        if  $(i, b+1) \notin Q$  then
17           $Q.insert((i, b+1), p(i, b+1) = d);$ 
18        if  $Q[(i, b+1)] \geq p + E(h+1, i)$  then
19           $Q.update((i, b+1), p(i, b+1) = d);$ 
20 return  $p;$ 
```

E Case Study

As a case study, we apply COMPACTBFS to the problem of *temporal community search* [Galimberti *et al.*, 2021]. Given a temporal graph $G_{\mathcal{T}}$ over a temporal domain $\mathcal{T} = [0, 1, \dots, t_{max}]$, an integer h , and a set of query nodes q , the problem seeks a partitioning P of the temporal domain into h segments and a subgraph G_h that contains the query nodes q within each bucket and maximizes the *sum of minimum degrees* of subgraphs in P . This problem is pertinent as the growing availability of timestamped data generates interest in temporal graphs. Real-world temporal graphs typically align themselves in evolving communities, which one may study by focusing on a set of query nodes.

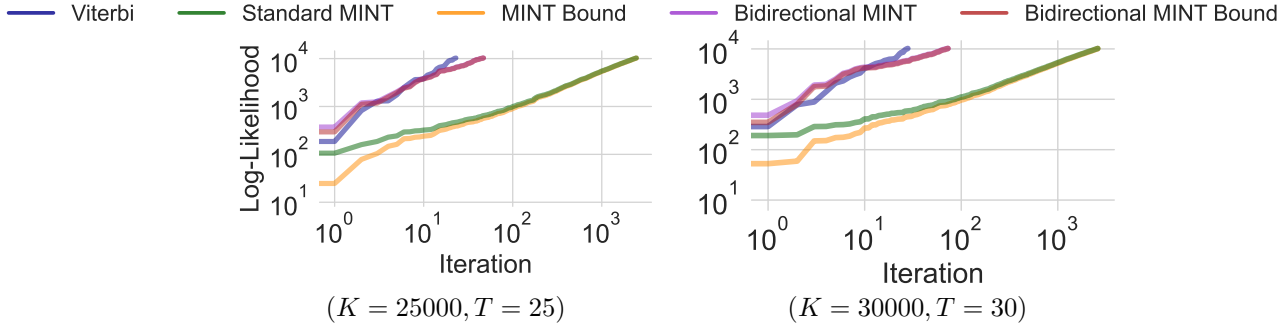


Figure 12: Synthetic data, skewed path likelihood distribution; path log-likelihood (in absolute value) on the fly; axes on log-scale.

Algorithm 4: DFS

Data: HMM graph G , transition and emission probabilities A and B , states S , observations Y , initial state s_h , predecessor pred , initial frame t , initial path priority p_h , middle pair, queue Q , middle frame.

Result: updated queue Q .

```

1 if  $t = \text{mid\_frame}$  then
2    $\text{mid\_pair} \leftarrow (\text{pred}, s_h)$ ; // update middle pair
3 if  $t < T$  then
4   for  $s_i \in \mathcal{N}_{\text{out}}(s_h)$  do
5     if  $s_i \in S$  then
6        $d \leftarrow p_h - \log A_{s_h, s_i} - \log B_{s_i, y_{t+1}}$ ;
7       if  $(s_i, t+1) \in Q$  then
8         if  $Q[(s_i, t+1)] > d$  then
9            $Q.\text{update}((s_i, t+1), d, s_h, \text{mid\_pair})$ ;
10        else
11           $Q \leftarrow \text{DFS}(\dots, s_i, s_h, t+1, d, \text{mid\_pair}, Q, \text{mid\_frame})$ ;
12 else
13   if  $(s_h, t) \notin Q$  then  $Q.\text{insert}((s_h, t), p_h, \text{pred}, \text{mid\_pair})$ ;
14   else  $Q.\text{update}((s_h, t), p_h, \text{pred}, \text{mid\_pair})$ ;
15 return  $Q$ ;

```

Algorithm 5: MINT-LS

Data: HMM graph G , transition and emission probabilities A and B , states S , observations Y , queue size threshold θ , initial and final state startSt and lastSt , initial and final frame startFr and lastFr .

```

1  $\text{mid\_frame} \leftarrow \lceil (\text{startFr} + \text{lastFr})/2 \rceil$ ,  $V \leftarrow \{\}$ ;
2  $Q \leftarrow \text{Queue}(\text{startSt}, \text{startFr}, d, -1, (-1, -1))$ ;
3 while  $Q \neq \emptyset$  do
4    $(s_h, t), p_h, \text{pred}, \text{mid\_pair} \leftarrow Q.\text{pop}()$ ;
5    $V.\text{add}((s_h, t))$ ;
6   if  $t = \text{mid\_frame} \wedge \text{mid\_pair} = (-1, -1)$  then
7      $\text{mid\_pair} \leftarrow (\text{pred}, s_h)$ ; // update mid pair
8   if  $(t = \text{lastFr}) \wedge (\text{lastSt} = -1 \vee \text{lastSt} = s_h)$  then
9      $s_{m-}, s_{m+} \leftarrow \text{mid\_pair}$ ; // extract mid pair
10     $N_p \leftarrow \text{mid\_frame}$ ; // frames before middle pair
11    if  $N_p > 1$  // continue in predecessors
12      then
13         $S_p \leftarrow \text{FIND-T-HOPRED}(s_{m-}, N_p)$ ;
14         $\text{MINT-LS}(G, A, B, S_p, Y, F, \theta, \text{startSt}, s_{m-}, \text{startFr}, N_p)$ ;
15         $N_s \leftarrow \text{startFr} + N_p$ ; // frames after middle pair
16        print  $(s_{m-}, s_{m+})$ ; // in-order print
17        if  $N_s > 1$  // continue in successors
18          then
19             $S_s \leftarrow \text{FIND-T-HOPSUCC}(s_{m+}, N_s)$ ;
20             $\text{MINT-LS}(G, A, B, S_s, Y, F, \theta, s_{m+}, \text{lastSt}, N_s, \text{lastFr})$ ;
21    for  $s_i \in \mathcal{N}_{\text{out}}(s_h)$  do
22      if  $(s_i, t+1) \notin V \wedge s_i \in S$  then
23         $d \leftarrow p_h - \log A_{s_h, s_i} - \log B_{s_i, y_{t+1}}$ ;
24        if  $Q[(s_i, t+1)] > d \vee (s_i, t+1) \notin Q$  then
25          if  $Q.\text{size}() > \theta \wedge (s_i, t+1) \notin Q$  then
26             $\text{DFS}(\dots, s_i, \text{pred}, t+1, d, \text{mid\_pair}, Q, \text{mid\_frame})$ ;
27          else
28            if  $(s_i, t+1) \notin Q$  then
29               $Q.\text{insert}((s_i, t+1), d, s_h, \text{mid\_pair})$ ;
30            else  $Q.\text{update}((s_i, t+1), d, s_h, \text{mid\_pair})$ ;

```

Algorithm 6: MINT-LS+

Data: HMM graph G , transition and emission probabilities A and B , states S , observations Y , queue size threshold θ , initial and final state startSt and lastSt , initial and final frame startFr and lastFr , maximum number of frames in the queue Δ_Q .

```

1  $\text{midFr} \leftarrow \lceil (\text{startFr} + \text{lastFr})/2 \rceil$ ,  $V \leftarrow \{\}$ ;
2  $Q \leftarrow \text{Queue}(\text{startSt}, \text{startFr}, 0, -1, (-1, -1))$ ;
3  $Q_t \leftarrow \text{Queue}(\text{startSt}, \text{startFr}, 0, -1, (-1, -1))$ ;
4  $\text{curr\_maxFr} \leftarrow \text{startFr}$ ,  $\text{EFS\_flag} \leftarrow \text{False}$ ;
5 while  $Q \neq \emptyset$  do
6   if  $\text{EFS\_flag} = \text{True}$  then
7      $(s_h, t), p_h, \text{pred}, \text{mid\_pair} \leftarrow Q_t.\text{pop}()$ ;
8      $Q.\text{delete}((s_h, t))$ ;
9   else
10     $(s_h, t), p_h, \text{pred}, \text{mid\_pair} \leftarrow Q.\text{pop}()$ ;
11     $Q_t.\text{delete}((s_h, t))$ ;
12     $V.\text{add}((s_h, t))$ ;
13    if  $t = \text{midFr} \wedge \text{mid\_pair} = (-1, -1)$  then
14       $\text{mid\_pair} \leftarrow (\text{pred}, s_h)$ ; // update middle pair
15    if  $(t = \text{lastFr}) \wedge (\text{lastSt} = -1 \vee \text{lastSt} = s_h)$  then
16       $s_{m-}, s_{m+} \leftarrow \text{mid\_pair}$ ; // extract middle pair
17       $N_p \leftarrow \text{midFr}$ ; // # frames before middle
18      if  $N_p > 1$  // continue in predecessors
19        then
20           $S_p \leftarrow \text{FIND-T-HOPRED}(s_{m-}, N_p)$ ;
21           $\text{MINT-LS+}(G, A, B, S_p, Y, F, \theta, \text{startSt}, s_{m-}, \text{startFr}, N_p, \Delta_Q)$ ;
22       $N_s \leftarrow \text{startFr} + N_p$ ; // frames after middle
23      print  $(s_{m-}, s_{m+})$ ; // in-order print
24      if  $N_s > 1$  // continue in successors
25        then
26           $S_s \leftarrow \text{FIND-T-HOPSUCC}(s_{m+}, N_s)$ ;
27           $\text{MINT-LS+}(G, A, B, S_s, Y, F, \theta, s_{m+}, \text{lastSt}, N_s, \text{lastFr}, \Delta_Q)$ ;
28    for  $s_i \in \mathcal{N}_{\text{out}}(s_h)$  do
29      if  $(s_i, t+1) \notin V \wedge s_i \in S$  then
30         $d \leftarrow p_h - \log A_{s_h, s_i} - \log B_{s_i, y_{t+1}}$ ;
31        if  $Q[(s_i, t+1)] > d \vee (s_i, t+1) \notin Q$  then
32          if  $(s_i, t+1) \notin Q$  then
33             $Q.\text{insert}((s_i, t+1), d, s_h, \text{mid\_pair})$ ;
34             $Q_t.\text{insert}((s_i, t+1), t+1, d, s_h, \text{mid\_pair})$ ;
35            if  $t+1 > \text{curr\_maxFr}$  then
36               $\text{curr\_maxFr} \leftarrow t+1$ ; // max frame
37          else
38             $Q.\text{update}((s_i, t+1), d, s_h, \text{mid\_pair})$ ;
39             $Q_t.\text{update}((s_i, t+1), t+1, p, s_h, \text{mid\_pair})$ ;
40    if  $\text{EFS\_flag} = \text{False}$  then
41      if  $Q.\text{size}() > \theta$  then
42         $\text{EFS\_flag} \leftarrow \text{True}$ ; // activate EFS
43    else
44       $(\text{curr\_min\_state}, \text{curr\_minFr}) \leftarrow Q_t.\text{top}()$ ;
45      if  $Q.\text{size}() < \theta \vee \text{curr\_maxFr} - \text{curr\_minFr} < \Delta_Q$  then
46         $\text{EFS\_flag} \leftarrow \text{False}$ ; // deactivate EFS

```

The problem is solved by the DP recursion:

$$p^*(i, b) = \max_{0 \leq j < t_{\max}} p^*(j, b-1) + v_q^*(j+1, i), \quad (5)$$

where $p^*(i, b)$ denotes the optimal objective value for a

partition of the first i timestamps in b segments and $v_q^*(j+1, i)$ is the maximum *minimum degree* of a subgraph containing query nodes q and enduring from the $(j+1)^{\text{th}}$ to the i^{th} timestamp. A cross-examination of Equations (5) and (4) reveals their analogy, with the main difference lying in the value associated with each segment, i.e., in the terminology of Section 3, the *gap function*. Thus, the dynamic-programming algorithm for histogram construction also solves temporal community search with the necessary modifications.

Nevertheless, to compute the gap function $v_q^*(j, i)$ we need to identify a subgraph containing the query nodes q of maximum minimum degree, for each query and each of the $O(t_{max}^2)$ possible (j, i) -buckets. The solution proposed by Galimberti *et al.* [2021] precomputes all gap function values through *span-core decomposition* and uses them in the dynamic-programming recursion of Equation (5).

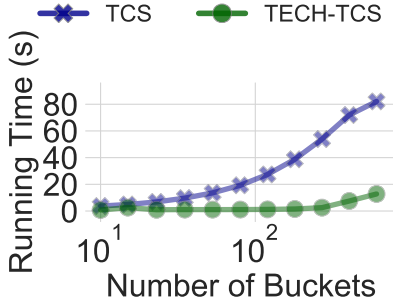


Figure 13: Temporal community search; runtime by # of buckets.

We apply COMPACTBFS to obtain an advantage over the DP solution to temporal community search on a real-world temporal network that captures interactions between students and teachers of nine high school classes in France over five days [Mastrandrea *et al.*, 2015]. The temporal graph has 47,590 edges (interactions) and 327 nodes (students and teachers). These parameters only affect the offline pre-computation of gap function values and not the query processing phase. The length of the sequence to be partitioned is $t_{max} = 1212$. We apply the standard DP algorithm (TCS) and an algorithm based on Standard TECH (TECH-TCS) on the problem with a query comprising the node labelled 1. Figure 13 plots runtime vs. the number of buckets that partition the temporal domain, varied in geometric progression with ratio 1.5, from 10 to 608. Notably, TECH-TCS outpaces TCS, even for a few buckets. We obtained similar results with different query nodes and larger query node sets, as the query does not affect the search space of the DP solution and its COMPACTBFS counterpart.